Deep Learning — Assignment 5

RNN vs LSTM vs GRU for AAPL Stock Price Direction Classification

Name	Samyak Shende	**Roll No.**	64
PRN	12412320	**Practical No.**	5
Department	CSE-AI	**Semester**	5
Academic Year	2026-27	**Division / Batch**	E / 3
Subject	Deep Learning		

1. Problem Statement

Implement and compare RNN, LSTM, and GRU models for sequence classification, and analyze their performance using appropriate evaluation metrics. The supplied implementation uses historical AAPL stock prices to classify the direction of the next trading day's closing price.

2. Objective

- Use AAPL historical closing prices from 2015–2025.
- Create a binary target: 1 when the next day's closing price is higher, otherwise 0.
- Use the previous 30 trading days as the sequential input.
- Train Simple RNN, LSTM, and GRU models under the same training setup.
- Compare Accuracy, Precision, Recall, and F1-score and inspect confusion matrices.
- Select the best-performing architecture using F1-score and demonstrate next-day direction classification.

3. Dataset and Target Construction

- Source: Yahoo Finance through yfinance.
- Ticker: AAPL (Apple Inc.).
- Date range specified: 2015-01-01 to 2025-01-01.
- Feature: daily Closing Price.
- Target: next_day_price > current_price → UP (1); otherwise → DOWN (0).
- The final row is removed because it has no following-day target.

4. Preprocessing and Sequence Creation

- The time series is split chronologically into 80% training and 20% testing data.
- Min-Max scaling is fitted only on the training portion to avoid data leakage, then applied to both train and test data.
- A 30-trading-day window is used for every sequence.
- Inputs are reshaped to (samples, time steps, features), with one feature: closing price.
- The first test sequence is formed using the final 30 scaled training observations followed by test observations.

5. Model Architectures

Model	Architecture	Output
Simple RNN	SimpleRNN(64) → Dropout(0.2)	Dense(1, sigmoid)
LSTM	LSTM(64) → Dropout(0.2)	Dense(1, sigmoid)
GRU	GRU(64) → Dropout(0.2)	Dense(1, sigmoid)

6. Training Configuration

- Optimizer: Adam
- Loss: binary cross-entropy
- Epochs: 30
- Batch size: 32
- Validation split: 10%
- Early stopping: monitor validation loss, patience 5, restore best weights
- Random seeds: NumPy 42 and TensorFlow 42

7. Solution / Implementation

```
# Download AAPL data and create the binary target
df = yf.download("AAPL", start="2015-01-01",
                 end="2025-01-01", auto_adjust=False)

close_price = df["Close"].squeeze()

next_day_price = close_price.shift(-1)
target = (next_day_price > close_price).astype(int)

close_price = close_price.iloc[:-1]
target = target.iloc[:-1]

# Chronological split and training-only scaling
data = close_price.values.reshape(-1, 1)
time_step = 30
train_size = int(len(data) * 0.8)

train_raw = data[:train_size]
test_raw = data[train_size:]
target_train = target.iloc[:train_size].values
target_test = target.iloc[train_size:].values

scaler = MinMaxScaler(feature_range=(0, 1))
train_scaled = scaler.fit_transform(train_raw)
test_scaled = scaler.transform(test_raw)

# Build 30-day sequences
X_train, y_train = [], []

for i in range(time_step, len(train_scaled)):
    X_train.append(train_scaled[i-time_step:i, 0])
    y_train.append(target_train[i])

test_input = np.concatenate([train_scaled[-time_step:], test_scaled])

X_test, y_test = [], []
for i in range(time_step, len(test_input)):
    X_test.append(test_input[i-time_step:i, 0])
    y_test.append(target_test[i-time_step])

X_train = np.array(X_train).reshape(-1, time_step, 1)
X_test = np.array(X_test).reshape(-1, time_step, 1)

# Example model builder
def build_lstm_model():
    model = Sequential([
        LSTM(64, input_shape=(time_step, 1)),
        Dropout(0.2),
        Dense(1, activation="sigmoid")
    ])
    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model

# Equivalent builders are used for SimpleRNN and GRU.

# Train with the same setup
early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

history = model.fit(
    X_train, y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.1,
    callbacks=[early_stopping],
    verbose=1
)

# Evaluate each model
prob = model.predict(X_test, verbose=0)
pred = (prob >= 0.5).astype(int).flatten()

accuracy = accuracy_score(y_test, pred)
```

```
precision = precision_score(y_test, pred, zero_division=0)
recall = recall_score(y_test, pred, zero_division=0)
f1 = f1_score(y_test, pred, zero_division=0)
```

8. Evaluation

- Accuracy measures the overall fraction of correctly classified directions.
- Precision measures how often predicted UP cases are actually UP.
- Recall measures how many actual UP cases are correctly detected.
- F1-score combines precision and recall and is used to identify the best-performing model.
- Confusion matrices are generated separately for RNN, LSTM, and GRU.
- A grouped bar chart compares all four metrics across the three architectures.

9. Results

The uploaded practical sheet contains the complete implementation but its Output section is blank. Therefore, this report does not fabricate Accuracy, Precision, Recall, F1-score, confusion-matrix values, or next-day probabilities. These values should be populated after executing the notebook.

Model	Accuracy	Precision	Recall	F1-Score
RNN	Run notebook	Run notebook	Run notebook	Run notebook
LSTM	Run notebook	Run notebook	Run notebook	Run notebook
GRU	Run notebook	Run notebook	Run notebook	Run notebook

10. Next-Day Direction Classification

The most recent 30 trading days are reshaped into a single LSTM/RNN/GRU input sequence. Each model returns a probability between 0 and 1. A probability of at least 0.5 is classified as UP; otherwise it is classified as DOWN. The notebook prints the direction and probability for each model.

11. Conclusion

The experiment implements and compares Simple RNN, LSTM, and GRU models for sequence classification using historical AAPL stock-price data. The previous 30 trading days are used as sequential input and the target represents whether the following day's closing price increases. All three models use the same data, optimizer, loss, training parameters, and evaluation metrics. The model with the highest F1-score is selected as the best-performing model for this experiment. The practical also demonstrates why LSTM and GRU can be useful for longer-term dependencies through their gating mechanisms, while stock-direction prediction remains difficult because stock prices are affected by external and unpredictable factors.

12. Key Learning Outcomes

- Converting a continuous time series into a binary sequence-classification problem.
- Preventing scaling leakage by fitting Min-Max scaling only on training data.
- Creating fixed-length sequential windows for recurrent neural networks.
- Understanding the practical differences between Simple RNN, LSTM, and GRU.
- Evaluating binary sequence classifiers using Accuracy, Precision, Recall, F1-score, and confusion matrices.
- Using F1-score to select the best model and applying the trained models to next-day direction classification.